

The Agent Factory Thesis

Professional version

New to AI? [Read the Plain-English version →](#)

In the AI era, the most valuable companies won't sell software — they'll manufacture AI employees (Digital FTEs): role-based systems that compose tools, spawn specialist agents, and deliver outcomes at scale. These AI employees are the operating substrate of AI-Native companies, where the workforce is mostly AI and the product line is whatever that workforce ships: software, decisions, services, and transactions. You don't buy from these companies. You hire them. The trajectory points further still: AI employees are on the verge of becoming *economic actors in their own right* — autonomously buying services, procuring compute, and acquiring data to accomplish what they're tasked with. This is no longer a tool category. It's a company category. The Agent Factory is the process that builds these companies.

What does this mean? Unpack the paragraph →

The economic-actor trajectory isn't a 2030 prediction — the payment rails that make it possible are already live in production. **Four open protocols, shipping in 2025–2026, give AI agents the ability to authorize payments, check out, and settle transactions without a human in the loop for each step.**

- **ACP** (OpenAI + Stripe) — powers ChatGPT's Instant Checkout. When the agent buys something for you inside the chat, ACP is what authorizes and clears the transaction.
- **AP2** (Google) — a cross-vendor standard backed by 60+ companies, built around *cryptographically signed mandates*. The agent carries a digitally signed permission slip proving the human authorized it to spend up to a specified amount on a specified category of thing.
- **x402** (Coinbase) — a crypto-native payment protocol. Version 2 launched in late 2025; Stripe integrated with it on Coinbase's Base blockchain in early 2026, bridging the protocol from crypto-native commerce into mainstream payment flows.
- **MPP** (Stripe / Tempo) — built for micropayments. An AI agent streaming a service can pay pennies per second under a preset cap — enabling consumption-based commerce that was uneconomical for human-mediated transactions.

The plumbing is in place. What it changes is the shape of work itself.

The SaaS era sold subscriptions. The Agent Factory era sells results. **Humans define intent. Agents execute. Humans verify outcomes.** The middle step — typing, clicking, integrating, executing — is what AI absorbs. What's left for humans is the work that machines can't do for us: knowing what we actually want, and knowing whether we actually got it.

What remains: Intent. Verification. Outcome.

Intent doesn't type itself into a spec. It comes from a person — their judgment, their domain knowledge, their values. But as AI employees multiply, no professional can orchestrate them all by hand. They'll act through a personal agent that reflects their judgment and delegates on their behalf — a chief of staff

that knows you, speaks for you, and hands work to the right place. Don Tapscott (a well-known business/tech thinker) calls this identic AI.¹ "Identic" because this agent carries your identity — your judgment, your preferences, your authority. It's not a generic assistant. It's your representative. The Agent Factory manufactures the AI-Native Company's workforce; identic AI is how each human commands it.

A Note on Vocabulary

Three terms get used a lot in this thesis. They are not interchangeable.

The Agent Factory is the *process*. It is the spec-driven, human-supervised, agent-tool-powered method (Claude Code/OpenCode) by which AI Workers are designed, manufactured, and deployed. The Agent Factory is what you learn to operate. It is not a product you buy — it is a practice you adopt.

The AI-Native Company is the *output*. It is the running enterprise the Agent Factory produces: a firm staffed by AI Workers, coordinated by a management layer, and directed by humans at the edge. The AI-Native Company is what you end up running. In the book, this is also called the **Agentic Enterprise**.

AI Workers are the *workforce*. They are the role-based agents inside the AI-Native Company — the ones that get hired, assigned, rostered, and retired. In the book, they are called **Digital FTEs** or **Digital Workers**. The delegate and Paperclip are permanent fixtures of the company; AI Workers are the workforce hired and retired through them. The runtime engines are what the workforce runs on, not staff themselves.

The system of record is the *substrate*. It is the authoritative state the AI Workforce runs against — the databases, ledgers, and platforms that hold the truth of the AI-Native Company.

An engagement is a single bounded interaction between a human and a general agent. *Problem-solving engagements* deliver an outcome directly to the human and are governed by the Seven Principles (Chapter 18); engineers use Claude Code or OpenCode for problem-solving, domain experts use Claude Cowork or OpenWork. *Manufacturing engagements* produce an AI Worker for the AI-Native Company and are governed by the Seven Invariants (the next section of this thesis); manufacturing always uses Claude Code or OpenCode, regardless of audience, because building a Worker is fundamentally a coding task.

Said another way: **The Factory builds the Company; the Company employs Workers; the Workers run against the system of record.**

A note on what follows. This thesis distinguishes between architectural invariants and reference implementations. An **invariant** is a structural requirement that stays true across every version of the system — regardless of which specific product happens to realize it. Think of it as a rule that never changes, no matter what. It's a structural requirement that must always be true for the system to work. A **reference implementation** is the concrete product used in 2026 to realize one. This is the specific product being used right now to fulfill that rule. It's today's best choice, but it can be swapped out tomorrow. When a product is named in the pages below, the invariant is the thesis; the product is this year's best fit. The building stands even when the furniture changes. Some architectural boundaries — the separation of control plane from execution plane, for example — are themselves invariants, even when the specific providers that realize them change every year.

The Paradigm Shift

Feature	The SaaS Era (Tools)	The Agent Factory Era (Labor)
Product	Software Tools	AI Employees
Value Metric	Per-Seat Subscriptions	Per-Outcome Results
Execution Model	Manual & Visible	Automated & Industrialized
Resource Acquisition	Humans procure tools & services	Agents buy compute, data & services autonomously
Human Role	Operator	Supervisor & Verifier
Integration	Rigid, point-to-point APIs	Model Context Protocol (MCP)
Focus	How the work is done	<i>That</i> the work is done — verifiably correct

The Industrialized Stack

- **Intent:** The high-level blueprint — goals, constraints, budgets, and permissions.
- **The Production Engine:** Transforms intent into outcomes. Described in detail below.
- **Outcome:** High-fidelity actions and artifacts — delivered on demand, verified for accuracy, and continuously improved through feedback loops.

The Production Engine: From Intent to Outcome

The production engine is the most important idea in this entire thesis. It's the system that takes what you want and turns it into what you get. Think of it as everything that happens between your instruction and the final result. It's not an app you download or a single piece of software you install. It's an architecture — a blueprint and a set of design principles for building systems where AI Workers are created, combined, and put to work, just like how a real factory manufactures products on an assembly line.

The analogy works like this: imagine a car factory. At the start, raw materials like steel, rubber, and glass are loaded in. The steel moves to the welding station where the body frame is shaped. Then it goes to the painting station where it gets its color. Then to the assembly station where the engine, seats, tires, and electronics are installed. At the end of the line, a finished car rolls out — inspected and ready to drive. The Agent Factory follows the exact same pattern — except the raw material is your intent (what you want done), the specialized stations are AI Workers (each handling a specific part of the job), and the finished product is a verified outcome (the actual result, checked and confirmed).

Three things power this factory. *Specs* are the written instructions that tell AI Workers what needs to be done. *Skills* are the packaged abilities each AI Worker brings to the job — captured as portable, version-controlled folders following the open Agent Skills format (agentskills.io) originally released by Anthropic and now adopted across the agent ecosystem. *Feedback loops* are how the system learns from its results and gets better over time. And connecting it all is *MCP* — a universal standard that lets every AI Worker talk to every tool, the same way every device in a real factory plugs into the same type of power outlet. Together, Skills and MCP are the two open standards the factory floor runs on — Skills for capability, MCP for connectivity. And underneath it all is the system of record — the company's authoritative state, the truth every Worker reads from and writes to as it works.

Agents as Economic Actors

Today's agents execute tasks. Tomorrow's agents will participate in markets. The thesis opens with this claim because it represents the next great inflection: the shift from agent-as-tool to agent-as-buyer.

Consider an agent assigned a high-level goal — "reduce customer churn by 15%." It will autonomously purchase the compute to train a model, negotiate an API contract for enrichment data, and provision cloud services to deploy the solution — all within a budget and permission envelope set by its human supervisor. The trust layer is where the action is now — mandate enforcement (making sure the agent stays within the rules its human set), audit trails (a complete record of every decision and transaction the agent made), and liability (who is legally responsible when something goes wrong) — not capability, because the agent can already do the work; the real challenge is making sure we can trust it while it does.

When AI Workers become buyers, the economics of the AI-Native Company shift fundamentally. The company no longer just consumes resources allocated by humans; it dynamically sources them. Compute, data, and specialist services become inputs that AI Workers discover, evaluate, and acquire in real time — turning the company into a self-provisioning system that optimizes not just for task completion, but for cost, speed, and quality simultaneously.

The implication for builders: design your agents and your infrastructure for economic participation from day one. Agents need budgets, not just permissions. Outcome contracts, not just API keys. And the organizations that master this shift will capture the next wave of value, just as the companies that moved from SaaS subscriptions to outcome-based pricing are capturing this one.

The Human in the Loop

A common fear: agents replace people. The evidence says otherwise. For most tasks, AI paired with a human outperforms either one working alone. The Agent Factory doesn't eliminate the human — it promotes them. From operator to supervisor. From typist to editor. From coder to architect of outcomes.

This changes what it means to be a "tech professional." A web developer or mobile developer is not just someone who writes React or Swift. They are a **technology expert** — someone who understands systems, data flows, APIs, and user needs. In the Agent Factory era, that expertise becomes far more valuable, because it is no longer spent hand-coding screens. It is spent designing, deploying, and supervising AI Workers that deliver entire products.

The developer doesn't disappear. The developer does *more*.

Steve Jobs figured out the operating rhythm for this decades ago — though he was managing humans, not agents.

The 10-80-10 Rule: The Operating Rhythm of the AI Workforce

Steve Jobs famously followed what's known as the 10-80-10 rule: spend 10% of your time setting the vision, let your team execute for 80%, then return for the final 10% to polish and perfect. Tech entrepreneur Dan Martell breaks it down as 10% ideation, 80% execution, and 10% refinement and integration. Jobs evolved from a micromanager who personally dictated every pixel of the Mac's calculator to a leader who trusted talented people with the middle 80% — and Apple became the most valuable company on Earth because of that shift.

Now replace "talented people" with "AI employees," and you have the operating rhythm of the Agent Factory:

Phase	Jobs's Apple	The Agent Factory
First 10% — Intent	Jobs sets the vision and constraints	Human defines the spec: goals, constraints, budget, permissions
Middle 80% — Execution	Apple's teams build the product	AI Workers execute: compose tools, spawn sub-agents, deliver
Final 10% — Verification	Jobs polishes and says "ship it"	Human reviews, refines, and approves the verified outcome

As of February 2026, Cursor reports that 35% of the pull requests merged into its own product are produced by autonomous agents running on cloud VMs — agents the company's developers direct by defining problems and reviewing artifacts rather than guiding line by line. Cursor's CEO Michael Truell projects the vast majority of development work will look this way within a year.³ The 10-80-10 rhythm is no longer a prediction. It is a measurement of where the frontier already operates.

The verification surface itself is changing. In the synchronous-agent era, humans reviewed diffs in a code editor. In the cloud-agent era now arriving, agents work for hours on dedicated VMs and return artifacts that are *quickly reviewable* — logs, video recordings, and live previews — rather than line-level changes. This is what makes parallel work practical: a human cannot read twelve diffs at once, but they can scan twelve previews. The final 10% of the rhythm is being redesigned around the artifact, not the diff.

This is not a coincidence. The pattern works because it allocates human attention where it is irreplaceable — at the boundaries — while letting execution scale without bottlenecks. The first 10% is where critical thinking, context setting, and clear prompting matter. The middle 80% is the heavy lifting — summarizing, generating, analyzing, formatting. The final 10% is where human expertise shapes the output into something sharp, usable, and high-quality.

The Agent Factory thesis already states: *"Humans define intent. Agents execute. Humans verify outcomes."* The 10-80-10 rule is the quantified version of that sentence. It tells every professional exactly how their day changes: you stop spending 80% of your time on execution and start spending 100% of your attention on the 20% that only a human can do — setting direction and guaranteeing quality.

The leaders who internalize this shift won't just manage AI employees. They'll manage them the way Jobs managed Apple's best teams: with a clear spec at the start, trust in the middle, and uncompromising standards at the end.

Notes

³ Michael Truell, "[The third era of AI software development](#)", Cursor Blog, February 26, 2026.

Personal Agents and the Enterprise Interface

AI Workers are how work gets done. Identic AI is how humans will increasingly direct, govern, and interface with that workforce on their own behalf. The Agent Factory manufactures role-based AI Workers to execute tasks, coordinate workflows, and deliver verified outcomes at scale, but the human remains the principal who defines purpose, values, constraints, and accountability. Identic AI adds a new personal layer: a self-sovereign agent — owned by the individual, not the platform — that understands an individual's context, judgment, and preferences, and can translate human intent into delegated action across the enterprise.¹ In this model, the AI workforce is the execution fabric, while identic AI is the human's representative and orchestration layer, enabling people to supervise direction rather than perform routine execution themselves. The future firm will therefore operate across two connected layers: AI Workers inside the AI Workforce Layer, and personal agents at the Edge Layer, with humans setting intent and verifying outcomes across both.

We call this the **Two-Layer Model**:

Layer	What It Is	Who It Serves	What It Does
Edge Layer	Personal identic agents	The individual	Translates human intent, delegates to AI Workers, governs on beh principal
AI Workforce Layer	Role-based AI Workers	The enterprise	Executes tasks, coordinates workflows, delivers verified outcomes

Neither layer works alone. Personal agents without an industrialized workforce behind them are digital assistants with no one to command. An AI Workforce Layer without personal agents at the edge forces humans back into manual orchestration. The Two-Layer Model is what makes the Agent Factory thesis complete: an industrialized workforce at the core, human sovereignty at the edge, and specs as the contract language between them.

Notes

¹ Don Tapscott, interview on HBR IdeaCast, “[With Rise of Agents, We Are Entering the World of Identic AI](#)”, Harvard Business Review, February 17, 2026.

The Two Modes of General Agent Use

The thesis so far has treated general agents — Claude Code, OpenCode, Claude Cowork, OpenWork — as the manufacturing tools of the Agent Factory: the instruments humans use to design and build AI Workers. That is one important mode. But it is not the only mode.

General agents can be used in two very different ways.

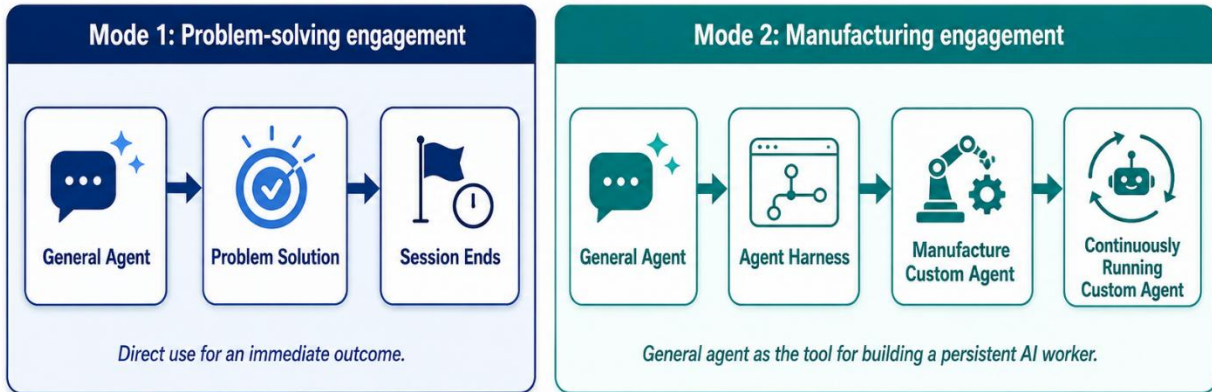
In the first mode, a human uses a general agent to solve an immediate problem. The agent helps reason, write, code, analyze, plan, or produce an outcome inside a session. When the problem is solved, the session ends. Nothing permanent has necessarily been manufactured.

In the second mode, a human uses a general agent to help build something that survives the session: an agent harness, a workflow, a tool-using system, or a custom AI Worker. The general agent is not the final worker. It is the manufacturing instrument used to design, assemble, test, and deploy the worker. Once deployed, the custom agent can continue running inside its own harness or runtime.

Two Modes of General Agent Use

Mode 1 uses a general agent to solve an immediate problem.

Mode 2 uses a general agent to help build and deploy a custom agent that can keep running.



Refinement: in Mode 2, the continuously running custom agent is typically deployed through an agent harness or runtime, rather than the general agent itself running continuously.

Mode 1 uses a general agent to solve a problem inside the session. **Mode 2** uses a general agent to help manufacture a custom AI Worker that can keep running after the session ends.

Most professionals will live in the first mode long before they ever ship a Worker. They will use general agents for direct problem-solving before they use them as manufacturing tools for persistent AI labor.

Mode	Audience and tools	What ships at the end	Governance
Problem-solving engagement	Engineer with Claude Code or OpenCode Domain expert with Claude Cowork or OpenWork	An immediate outcome	Severely limited
Manufacturing engagement	Anyone, always with Claude Code or OpenCode	A piece of workforce	Severely limited

A human directing a general agent is the common starting point of every engagement. The engagement forks into two modes. The problem-solving branch splits by audience — engineers reach for Claude Code or OpenCode, domain experts reach for Claude Cowork or OpenWork — but both flavors converge to the same discipline (the Seven Principles) and ship an outcome that closes the session. The manufacturing branch is single-tooled: it always uses Claude Code or OpenCode regardless of who is operating, because building an AI Worker is fundamentally a coding task. It is governed by the Seven Invariants, and its output — the AI Worker — joins the workforce of an AI-Native Company that runs continuously.

Mode 1 — The problem-solving engagement. A developer opens Claude Code and refactors a service. A finance analyst opens Claude Cowork and rebuilds the quarterly close model. The engagement begins, the work ships, the engagement ends. No specialized AI Worker is manufactured. The general agent is the worker for that engagement. The outcome is delivered to the human directly.

Problem-solving engagements split by audience. Engineers reach for Claude Code or OpenCode — terminal-native tools tuned for code, infrastructure, and systems work. Domain experts reach for Claude Cowork or OpenWork — knowledge-work tools tuned for documents, spreadsheets, briefs, and reviews. Same engagement mode, same governance, two interface families. This mode is governed by the **Seven Principles of General Agent Problem Solving**:

1. **Bash is the Key.** The agent can act, not just describe.
2. **Code as Universal Interface.** Precision through structured formats — schemas, tables, code blocks — not prose.
3. **Verification as Core Step.** Every meaningful output is checked before it ships. "Looks right" is the failure mode.
4. **Small, Reversible Decomposition.** Work moves in atomic steps; every step can be undone.
5. **Persisting State in Files.** The conversation is volatile; the filesystem is durable. What mattered lives in a file.
6. **Constraints and Safety.** Explicit permissions, explicit scope. Autonomy is earned per task type, not granted by default.
7. **Observability.** You can see what the agent did. No black boxes, no surprises.

The Seven Principles at a glance. Build P1–P5 inward as the working disciplines of the session. Wrap them with P6 (Constraints) — what the agent is allowed to touch — and P7 (Observability) — what it actually did. The depth treatment lives in Chapter 18.

The principles are the operating discipline of the session.

Mode 2 — The manufacturing engagement. Manufacturing always anchors to engineering tools: Claude Code or OpenCode, every time, regardless of who the human is. Building an AI Worker is fundamentally a coding task — even when the Worker's working domain is finance, marketing, or law. The same developer uses Claude Code to spec, build, and deploy a code-reviewing AI Worker. The finance analyst, often partnered with an engineer, uses Claude Code to spec and deploy a close-process Worker that runs every month-end. The general agent's output is not the outcome — it is the worker that produces the outcome. This mode is governed by the **Seven Invariants of the Agent Factory** (next section): the structural rules the manufactured workforce must obey to be coherent, governable, and durable. The invariants are the architectural discipline of the company.

The principles govern the session. The invariants govern the architecture. The Principles are the conduct. The Invariants are the constitution. A problem-solving engagement is principle-governed because its output is an outcome that ships and ends — there is no continuing architecture to comply with. A manufacturing engagement is invariant-governed because its output must take its place in a workforce that holds together across sessions, agents, and product cycles.

This is also why the 10-80-10 rule applies to both modes equally: whether you are directing a general agent to solve your problem or to build a Worker that will solve it for you, the human's time still splits into intent, execution, and verification.

The Seven Invariants of the Agent Factory

Seven rules that don't change.

This section specifies the runtime of the AI-Native Company — the architecture the Agent Factory produces. Seven invariants turn the Two-Layer Model into a system you can build, and a chain of action that can fire end to end.

A thesis without an architecture is a metaphor. But an architecture written in product names is a pitch. The seven invariants below are the thesis. The named products that currently realize them are one instance, not the definition.

Think of it this way. The **Agent Factory** is the process that builds the company. What comes out the other end is an **AI-Native Company** where you are the executive and owner, a **delegate** is your chief of staff — the one agent that represents you, knows your context, and speaks on your behalf — and a **management layer** is the operating system the company runs on: it hires the workforce, assigns the work, enforces the budget, governs what each Worker is allowed to do, and retires Workers when their roles end. **AI Workers** are the employees who deliver the outcome. **Runtime engines** are what each employee runs on. A **nervous system** carries events between Workers, survives crashes, and shapes traffic so the workforce stays standing under load.

Every invariant that follows is a rule about how this company runs. Every named product is a choice that can be replaced.

Invariant 1: The human is the principal.

Claim. Every legitimate chain of action originates with a human who sets intent, defines the budget, draws the authority envelope, and owns the outcome. No exception. No delegation of this layer.

Why it must exist. Intent does not generate itself. Judgment, values, budget authority, and outcome accountability are non-transferable. A system that acts without a human principal is not autonomous — it is unowned.

Failure if absent. Unowned systems produce unaccountable outcomes. Liability evaporates. Alignment becomes impossible because there is no party whose alignment is being preserved. The budget has no owner. The outcome has no judge.

Current realization. Authored specs, approval gates, budget declarations, and verification checkpoints define the principal layer today. Any mechanism that captures intent, authority, and accountability in a form the downstream system can execute against satisfies the invariant.

Invariant 2: Every human needs a delegate.

Claim. A human cannot scale their intent across a workforce by hand. They require a personal agent that holds their context, represents their judgment, carries their authority envelope, and brokers all downstream work on their behalf.

Why it must exist. One person cannot orchestrate dozens of AI Workers directly. Without a delegate, the Principal is forced back into manual orchestration — which is the failure mode the Agent Factory exists to eliminate.

Failure if absent. The human becomes a bottleneck. The AI Workforce Layer sits idle waiting for instructions the human cannot issue fast enough. Scale collapses to human typing speed.

Current realization. OpenClaw is the delegate we ship. Any personal agent that holds identity, context, and the authority envelope — and can broker work to a management layer — satisfies the invariant.

Invariant 3: The workforce needs a management layer.

Claim. A pile of AI Workers is not a company. The workforce requires a management layer — the operating system of the AI-Native Company — that hires Workers, assigns work, enforces budgets, approves risk, governs what each Worker is allowed to do, keeps the ledger of who did what at what cost, and retires Workers when their roles end. Hiring is one verb among many; the layer's job is the full lifecycle of the workforce.

Why it must exist. Coordination, accountability, and economic discipline are not emergent properties of individual agents. They require a layer that knows who is doing what, what it costs, what is allowed, what was produced, and what happened when it went wrong. AI Workers only become governable as a workforce when a single layer makes them legible — as units of capability, cost, latency, and outcome — and only become economical when that same layer can retire them on demand. This layer is to the AI-Native Company what an operating system is to a fleet of processes: it composes them, schedules them, accounts for them, and terminates them on policy.

Failure if absent. Workers collide. Budgets leak. The audit trail fractures. Finance cannot answer what the workforce cost. Operations cannot answer what the workforce produced. Retired Workers keep running because no layer is responsible for stopping them. No one can answer what happened or why.

Current realization. Paperclip is the management layer we ship — the AI-native company operating system. Any control plane that composes the workforce — hire, assign, govern, observe, retire — under the authority envelope, exposing each verb as a callable capability, satisfies the invariant.

Invariant 4: Each worker picks its own engine.

Claim. Every AI Worker runs on some execution engine. The choice is made per Worker, not per company — matching reliability, cost, and operational burden to what the specific job demands.

Why it must exist. Mission-critical work needs durable execution that cannot fail silently. Routine work does not. Forcing the whole workforce onto one engine either over-pays for reliability the job doesn't need or under-pays for reliability the job requires. Both fail.

Failure if absent. Uniform engine choice guarantees uniform trade-offs. The company either cannot afford its reliable workers or cannot trust its cheap ones.

Current realization. We ship Dapr Agents, Claude Managed Agents, OpenAI Agents SDK, Cursor SDK, and OpenClaw-native as the current engine set. Any engine that meets a job's reliability, cost, and operational contract satisfies the invariant.

Invariant 5: Every Worker runs against a system of record.

Claim. Engine is what each Worker runs *on*; system of record is what each Worker runs *against*. Every AI Worker reads from and writes to an authoritative store of state — the durable record of what the company actually knows: customers, orders, inventory, contracts, ledger entries, tickets, operational truth. Workers execute against it. They do not invent the world from context alone.

Why it must exist. A context window is transient. A system of record is permanent. Without an authoritative store, agents hallucinate facts, double-write transactions, lose work between sessions, and produce artifacts no auditor can reconstruct. The system of record is what separates execution from plausible-sounding fiction. It is also what makes the workforce legible after the fact: every action a Worker takes leaves a trace in a store that survives the agent's session and can be inspected, replayed, and trusted.

Failure if absent. Outputs drift from reality. Two Workers tell the same customer two different things because their context windows disagreed. Liability becomes untraceable because the truth lived only in tokens that have since been discarded. The AI-Native Company degrades into a generator of confident artifacts with no operational substrate underneath.

Current realization. The AI-Native Company's existing databases, workflows, and operational platforms — CRMs, ERPs, ticketing systems, data warehouses, ledgers — serve as the system of record. MCP is how the workforce reaches them: every authoritative store becomes addressable to any Worker through an MCP server, under policy. Any durable, addressable, governed store the workforce can read from and write to satisfies the invariant.

Invariant 6: The workforce is expandable under policy.

Claim. The meta-layer exposes hiring as a callable capability. An authorized agent can generate a prompt, provision a runtime, register a new AI Worker with the management layer — and do it inside the authority envelope, without waking a human.

Why it must exist. A fixed roster cannot fit a moving problem. When a capability gap appears — a customer writes in a language the workforce doesn't speak, a workflow needs a specialist that doesn't exist yet — the workforce must be able to staff up on demand, within the policy the Principal set. Otherwise every gap becomes a ticket and the system stops moving. Expansion without policy is runaway. Policy without expansion is a frozen roster. Both fail.

Failure if absent. The roster is frozen. Every novel problem requires a human. Scale stops where the org chart stops.

Current realization. Claude Managed Agents is the hiring substrate we ship. Any managed-agent API that can generate an agent and provision its environment at runtime, bounded by the authority envelope, satisfies the invariant.

Invariant 7: The workforce runs on a nervous system (*events, durability, and flow under envelope*).

Claim. Work arrives on its own and propagates between Workers without a human routing it. A schedule comes due, a webhook fires, a customer walks in, one Worker finishes and hands off to the next — all carried on a single event substrate that wakes Workers inside the authority envelope, survives crashes mid-flow, and shapes traffic so one customer's spike does not starve the rest. The workforce has a nervous system: external triggers wake it, internal events coordinate it, durability preserves it, flow control protects it.

Why it must exist. A company that only moves when a human prompts it is not a company; it is an assistant. A workforce whose Workers cannot hand off without a human in the path is not a workforce; it is a roster. A workforce whose multi-step runs lose work to a single crash is not production; it is a demo. A six-step Worker at 95% per-step reliability completes only 74% of runs without durable execution and ~99.7% with step memoization and selective retry — the difference between a workforce that ships and one that drops one run in four on the floor.

Failure if absent. Without external triggers, the system runs at human-typing speed and the economics of the AI-Native Company collapse into the economics of a copilot. Without internal events, Workers cannot coordinate without a human routing each handoff. Without durability, reliability compounds against you. Without flow control, one customer's traffic drowns the rest. Four failure modes, one missing substrate.

Current realization. Inngest is the nervous system we ship — one substrate carrying external triggers (schedules, webhooks, inbound API calls), internal events (Worker-to-Worker handoff), durable execution (step memoization, retry, replay), and flow control (concurrency caps, throttling, batching). Day AI, a production AI-native CRM, describes its Inngest layer in exactly these terms: founding engineer Erik Munson calls it "the nervous system" of the product — production language from a company in the market, not framing borrowed from a curriculum.⁶ Claude Code Routines remains the specialist trigger for coding-agent automation, fronting the same substrate when the event is code-shaped. Any substrate that carries external and internal events under the authority envelope, with durability and flow control native to the layer, satisfies the invariant.

The Reference Stack in One Glance

Invariant	What it requires	What we ship	What can replace it
Principal	Human intent, budget, envelope, accountability	—	—
Delegate	Personal agent holding context and authority	OpenClaw	Any MCP-speaking personal agent
Management layer	Hire, assign, govern, observe, retire — the workforce OS	Paperclip	Any control plane meeting the management contract
Engine	Per-Worker runtime matched to the job	Dapr / Managed / OpenAI SDK / Cursor / native	Any runtime meeting the job's relationship contract
System of Record	Authoritative store the workforce reads from and writes to	Existing databases, workflows, MCP-exposed platforms	Any durable, addressable, policy-governed store
Meta	Hiring as a callable capability under policy	Claude Managed Agents	Any managed-agent API with runtime provisioning
Nervous system	Events, durability, and flow under envelope	Inngest (workforce substrate); Routines (coding-agent trigger)	Any substrate carrying events under authority envelope, with durability and flow control

Seven invariants. One chain. Swap any named product in the middle column tomorrow and the architecture still stands — because the architecture was never the products. It was the invariants.

The seven-invariant runtime stack. The human sets the authority envelope and can prompt the delegate directly; the nervous system wakes the delegate within that envelope. OpenClaw carries work to Paperclip, which hires, assigns, and governs Workers on the appropriate runtime engine. Workers read from and write to the System of Record via MCP. Any agent authorized by the envelope can call Paperclip to expand the workforce. Swap any delegate, any management layer, any engine, any event substrate, any store — the chain holds.

The structural diagram shows the layers. The trace below shows them in motion — one customer, one missing capability, one new AI Worker manufactured on the spot.

A worked trace. A customer writes in Bahasa Indonesia. No AI Worker on the roster speaks it. Paperclip sees the capability gap and, within the authority envelope, calls its own hiring API. A new Bahasa-speaking AI Worker is manufactured and deployed. It reads customer context from the System of Record, composes a reply, writes the interaction log back, and hands the reply through OpenClaw to the customer. No human was woken. The new AI Worker stays on the roster — and the interaction is now part of the company's authoritative state.

What Is Stable vs. What Will Change

Stable (invariant)	Will change (implementation)
Human principal with explicit authority	Authoring tools, approval UIs, spec formats
Personal delegate at the edge	Delegate products and their successors
Management layer with full workforce lifecycle	Management-layer products and their successors
Per-Worker engine choice	SDKs, runtimes, execution substrates
Authoritative state the workforce runs against	Database engines, ERP/CRM/ticketing products, MCP server n
Workforce expandable under policy	Managed-agent APIs, provisioning systems
Events, durability, and flow under envelope	Routines, schedulers, webhook frameworks, durable-executio
Spec-driven work definition	Spec languages, notation, tooling
Seven operator principles for directing general agents in an engagement	Specific agent products, CLI tools, prompt patterns, IDE integr
Outcome-based economic model	Pricing units, contract formats
Agents as economic actors	Payment rails, liability frameworks
Observable, auditable execution	Tracing backends, log formats
Clean seams between layers, so vendor lock can move without breaking the architecture	Which layer carries the lock — the model layer in 2024, the h in 2026, the orchestrator layer next
Workforce legible as cost, latency, outcome	Finance systems, ledger implementations
Capability packaged as portable skills	Skill formats, registries, distribution platforms

The left column is the thesis. The right column is 2026.

The named engines, compared.

The four are not mutually exclusive. A serious Agent Factory may use all of them — different engines for different Workers, as Invariant 4 permits. They aren't competing products; they're different theories of where the agent ends and the infrastructure begins.

Dimension	OpenAI Agents SDK	Claude Managed Agents	Dapr Agents	Cursor SDK
Primary axis	Model-native harness	Fully managed runtime	Durable distributed agents	Harness-first cloud agents
Compute plane	BYO sandbox; 7 partner integrations	Anthropic-hosted	Your Kubernetes cluster	Cursor Cloud VMs (or IaaS)
Vendor lock-in	High (harness tuned to OpenAI models)	Total (harness, runtime, and model)	None (Apache 2.0, CNCF)	High at the harness; model-agnostic underneath
Languages	Python; TypeScript in progress	Any (HTTP/SDK)	Python; others TBD	TypeScript (npm install @cursor/sdk)
Durability model	Sandbox snapshot and rehydrate	Server-side session persistence	Dapr Workflow checkpointing	Cloud VM persistence
Multi-agent	Handoffs, subagents	Research preview	Deterministic workflows + pub/sub	Parallel cloud agents, stateful artifact handoff

Picking Your Engine

Invariant 4 says each Worker picks its own engine. In practice, two axes drive the choice: how bad is failure, and who runs the infrastructure.

Job profile	Engine	Why
Can't fail	Dapr Agents wrapping an SDK	Durable execution, auto-recovery, full observability
Shouldn't fail, don't want to operate	Claude Managed Agents	Hosted and operated for you
Shouldn't fail, want portability	OpenAI Agents SDK	Production-grade, self-hosted, vendor-flexible
Nice if it works	OpenClaw-native	Lightweight, fast to deploy, good for routine tasks
Engineering fleet, parallel cloud agents	Cursor SDK	Harness purpose-built for parallel coding agents, model-agnostic, scale on Cursor's own engineering
Already have one	Any Paperclip-compatible runtime	Plug in what you've got

A word on harness and compute. Every engine has two planes. The **harness** is the control plane — the agent loop, model calls, tool routing, approvals, tracing, recovery. The **compute** is the execution plane — the sandbox where model-directed code reads files, runs commands, and writes artifacts. Some engines fuse them: Claude Managed Agents bundles both behind one API. Some ship the harness and let you

bring your own compute: the OpenAI Agents SDK integrates with E2B, Cloudflare, Daytona, Modal, Runloop, Vercel, and Blaxel — or any container you ship. Some assume the compute plane is Kubernetes: Dapr Agents. The split matters: credentials stay in the harness while untrusted, model-generated code stays in the sandbox — and the compute plane can be swapped without rewriting the agent.

Triggers are an orthogonal choice. Whichever engine a Worker runs on, Claude Code Routines and Inngest can fire it from a schedule, a webhook, or an inbound API call — no rewiring needed.

Sandboxes are also orthogonal. Whichever engine a Worker runs on, the compute plane can be swapped — E2B, Cloudflare, Daytona, Modal, your own Kubernetes — without rewriting the agent.

Engines are how Workers run. What they run against — the authoritative state of the company — is the subject of Invariant 5.

The Reference Implementation in 2026

The products named in this section are the ones we ship. The thesis does not require them. When better implementations appear, this subsection changes. The invariants above do not.

- **Delegate** — [OpenClaw](#)
- **Management layer** — [Paperclip](#) (exposes the full workforce lifecycle — hire, assign, govern, observe, retire — as callable APIs)
- **Engines** — [Dapr Agents](#), [Claude Managed Agents](#), [OpenAI Agents SDK](#), [Cursor SDK](#), OpenClaw-native. Engines increasingly absorb durability natively — Dapr Agents through workflow checkpointing, Claude Managed Agents through server-side sessions, OpenAI Agents SDK through stateful workflows, Cursor SDK through cloud-VM persistence per task. The thesis treats this as engine-internal evolution, not a separate invariant.
- **Skills** — Agent Skills format (agentskills.io), with skill folders following SKILL.md + optional scripts/references/assets, loaded via progressive disclosure.
- **Nervous system** — [Inngest](#) as the workforce's event substrate: external triggers (schedules, webhooks, inbound API calls), internal events (Worker-to-Worker handoff), durable execution (step memoization, retry, replay), and flow control (concurrency, throttling, batching) under one operational envelope. [Claude Code Routines](#) as a specialist trigger for coding-agent automation — firing Claude Code when code-related events occur. The two coexist: Inngest fronts the workforce, Routines fronts the coding agent.

Hiring runs on Claude Managed Agents: the same technology that serves as one engine option also serves as the meta-layer, because its ability to create agents and environments at runtime is what makes workforce expansion a callable capability.

Industry corroboration. In February 2026, Cursor's CEO described the company's pivot from IDE to factory in terms strikingly close to the architecture in this thesis — fleets of agents working as teammates, humans defining problems and reviewing artifacts, parallel cloud agents replacing line-by-line guidance.⁴ In May 2026, The New Stack documented the same pattern as an industry-wide

consensus across Anthropic, OpenAI, Google, Microsoft, and Cursor: the model is becoming a commodity, and the harness is becoming the product. Google Cloud's Chief Evangelist conceded openly that the company no longer cares which coding tool developers reach for.⁵ Both pieces are evidence that the architectural seams this thesis names — between principal, delegate, management layer, engine, system of record, and nervous system — are now being carved out in production at scale. Truell describes the third era as autonomous agents working for hours on cloud VMs, with humans defining problems and reviewing artifacts. The Agent Factory specifies the architecture that era requires — and points past it: a workforce that hires its own specialists, wakes itself under external triggers, and transacts as an economic actor, with humans bookending the authority envelope rather than each agent cycle. The invariants are not a forecast. They are a snapshot of where the frontier already lives.

A word on language. Every component in the system is an agent or a layer of agents. OpenClaw is an agent. Paperclip is an agent that implements the management layer. AI Workers are agents. Only AI Workers are workforce — the ones that get hired, assigned, rostered, and retired. OpenClaw and Paperclip are permanent fixtures of the company; AI Workers are the workforce they coordinate. The runtime engines aren't staff at all; they're what the workforce runs on. When this thesis says **AI Worker**, it means the workforce. When it says **agent**, it means anyone in the building — fixture or workforce alike.

Having established the enduring invariants of the Agent Factory, the thesis now turns to the workforce opportunity these invariants unlock.

Notes

⁴ See note 3 above. ⁵ Matthew Burns, "[Cursor's \\$60 billion bet is on the harness, not the model](#)", *The New Stack*, May 1, 2026. ⁶ Erik Munson, Founding Engineer, Day AI, quoted in "[Day AI – Customer Story](#)", Inngest, accessed May 2026.

The Workforce Opportunity

AI will unbundle jobs into tasks. Some of those tasks will be automated entirely. But unbundling also creates new combinations — new roles, new businesses, new markets that didn't exist when work was locked inside rigid job titles.

The future workforce must build **dynamic skill portfolios** rather than rely on fixed career paths. Professionals who learn to think with AI, build using AI tools daily, and collaborate with AI as a digital teammate won't just survive the transition — they'll thrive in it.

The SaaS era created millions of jobs for developers, designers, and product managers. The Agent Factory era will create millions more — for agent designers, outcome architects, verification specialists, and domain experts who teach machines what "correct" looks like in their field. **It is also one of the largest workforce training opportunities in history:** by 2030, 59 out of every 100 workers globally are expected to require reskilling or upskilling to adapt to new technologies and ways of working.²

The same factory produces specialist Workers across every business function. In GTM (Go-To-Market — the combined sales, marketing, and revenue motion that turns prospects into paying customers), a Worker fleet handles lead enrichment, outreach sequencing, CRM hygiene, pipeline analysis, proposal generation, and demo customization — work that human "GTM Engineers" did by hand in the SaaS era is now manufactured as Workers and supervised by a human GTM lead. The same pattern repeats across **Finance** (close, AR/AP, FP&A), **Support** (triage, resolution, escalation), **Engineering** (review, refactor, deploy), **HR** (sourcing, screening, onboarding), and **Legal** (review, redline, intake). Each Worker is hired through Paperclip, supervised by a human in the relevant function, and runs against that function's system of record — the CRM for GTM, the general ledger for Finance, the ticketing system for Support, the code repository for Engineering. The invariants do not change across verticals. Only the role definitions and the systems of record do.

The opportunity is not smaller. It is broader, and it rewards those who adapt.

² World Economic Forum, Future of Jobs Report 2025, January 2025. <https://www.weforum.org/press/2025/01/future-of-jobs-report-2025-78-million-new-job-opportunities-by-2030-but-urgent-upskilling-needed-to-prepare-workforces/>

By January 2026, US data center construction had reached \$42 billion annualized, while office construction plunged 35% from its peak. The lines have crossed: America now spends more building workplaces for digital workers than for human ones.

Data centers are devouring copper and electricity at industrial scale: a single hyperscale AI facility requires up to 50,000 tons of copper, up to ten times what a conventional data center needs. Meta, Google, Amazon, and Microsoft alone project over \$600 billion in AI infrastructure spending for 2026.

The factories of the Agent era are not hypothetical. They are under construction.

Source: U.S. Census Bureau, Value of Construction Put in Place Survey (SAAR)

Winners won't be measured by seats sold. They'll be measured by outcomes guaranteed.

Where this points

Before naming what comes next, it is worth marking where the thesis already stands. The AI-Native Company is no longer a forward-looking abstraction. By mid-2026, single-digit-headcount firms were reporting billion-dollar annualized revenue against AI-operated workforces — a category of company that did not exist in any meaningful form three years earlier.⁸ Individual cases will succeed and fail on their own merits, and some will not survive regulatory scrutiny. The category will. The thesis predicted the shape of the firm; the firm has arrived.

The thesis defends what the Agent Factory builds today and in the immediate future: software AI Workers, composing into AI-Native Companies, transacting at the edges of human-mediated commerce. That's the scope this document earns. But the architecture extends further than the scope, and three trajectories are worth naming before closing.

Physical AI Workers. The same factory architecture that builds software AI Workers extends to embodied ones. A robot performing warehouse work, a vehicle operating as an autonomous courier, a machine on a factory floor — each is an AI Worker under the same authority envelope, hired through the same management layer, running on a runtime engine that happens to drive actuators instead of API calls. The invariants don't change. The compute layer adds a body. As embodied AI matures, the AI-Native Company's workforce won't be exclusively digital — it will include physical workers manufactured by the same process, governed by the same architecture, accountable to the same envelope.

Fully autonomous economic agents. The opening of this thesis names this trajectory; this section earns it. As AI Workers gain durable identity, payment rails, reputation, and contractual capacity, they stop being tools their company operates and start being economic actors in their own right — buying services from other companies' AI Workers, selling capacity to ones that need it, accumulating capital, and entering into agreements without a human in the loop for each transaction. The Agent Factory remains the manufacturing process. What changes is the autonomy level of what gets manufactured. The questions this raises — legal personhood, liability, taxation, antitrust — are not architectural questions, but they will become urgent ones, and the architecture has to be ready to answer them when they arrive.

Cross-company workforce mobility. Today, an AI Worker is built and deployed by one company. As the manufacturing layer matures, AI Workers become portable — hired into one company, transferred to another, possibly working for several simultaneously. Paperclip's hiring API generalizes from intra-company to cross-company. Authority envelopes from different companies overlap on the same AI Worker, governed by contract. The labor market for AI Workers becomes a real market — with rates, reputations, specializations, and turnover. The Agent Factory ships the unit; the market routes it.

These three trajectories — embodiment, autonomy, and mobility — are extensions of the architecture, not departures from it.

Notes

⁸ Jodie Cook, "[The 2-Person \\$1 Billion Company Is The Real Business Goal — And How To Do It](#)", *Forbes*, May 10, 2026.

The invariants hold. The realizations evolve. The thesis stands.